Private Stablecoin on Ethereum

PLabs

October 9, 2026

Abstract

We present an issuance protocol for private stablecoins on Ethereum: anyone can deploy a `ReserveVault` and `NotePool` backed 1:1 by a chosen reserve ERC-20. Instantiated with USDC (**pUSDC**), the user's first on-chain credit is a private note—without first holding public USDC. Holdings are pERC20 notes and do not replace the underlying reserve. Ownership and transfers are hidden from the chain. Circulating supply is contract-enforced 1:1 against reserves, not issuer disclosure.

Table 1: USDC vs. the USDC-backed protocol instantiation pUSDC (pUSDC column highlighted)

Dimension	USDC	pUSDC
Privacy after deposit	Public address balance	**Private note; no public address required**
Holders / balances / transfer graph	Public	**Hidden from the chain**
At mint	Immediately transferable and redeemable	Inventory; neither transferable nor redeemable
Circulating ↔ reserves	Issuer disclosure and audits	**Contract-enforced 1:1; on-chain checkable**
Redemption	Through the issuer	**Holder self-proves; permissionless**
Blocking funds	Issuer can freeze	Compliance freeze only
Disclosure	Fully public ledger	Selective; viewing keys

The design rests on two separations. **Payer ≠ recipient:** the party that locks the reserve token is an on-ramp, desk, or any third party; the party that receives the note is the user. **Minting ≠ becoming money:** pre-minted notes belong to a protocol address and cannot be transferred or redeemed; only after an equal reserve deposit in the same transaction do they become spendable, redeemable circulating claims. In general the hard invariant is reserve-token balance $\geq$ `circulating`; in the pUSDC instance it is `ReserveVault.USDC` $\geq$ `circulating`. Fiat on- and off-ramps still trust the ramp. On-chain contracts are immutable and trust neither the operator, the ramp, nor relayers. The sole on-chain privilege is a compliance freeze.

Keywords: private stablecoin; issuance protocol; permissionless deployment; Ethereum; zero-knowledge proofs; pERC20; day-one privacy; trustless issuance; trustless backing

Contents

1 Introduction **3**
- 1.1 The problem . . . 3
- 1.2 Core ideas . . . 3
- 1.3 Contributions . . . 3
- 1.4 Organization . . . 4

2 Related work **4**
- 2.1 Shielded pools on Ethereum: public coin first, privacy later . . . 4
- 2.2 Private stablecoins on non-Ethereum chains . . . 4
- 2.3 Comparison and Ethereum constraints . . . 4

3 System overview **5**
- 3.1 Architecture . . . 5
- 3.2 Two kinds of notes . . . 6
- 3.3 Trust and goals . . . 6

4 Protocol **7**
- 4.1 Notes, actions, and signatures . . . 7
- 4.2 Entry points and authorizations . . . 7
- 4.3 Circuit extensions: CP and CT . . . 8
- 4.4 Π_{buy}: purchase (main on-ramp path) . . . 8
- 4.5 Π_{transfer} and Π_{redeem} . . . 9
- 4.6 Responsibility matrix . . . 9
- 4.7 Building Private DeFi . . . 9
- 4.8 Composition with public DeFi . . . 9

5 Security and privacy **10**
- 5.1 Propositions and attack matrix . . . 10

6 Compliance and auditability **12**

7 Discussion and limitations **12**

8 Conclusion **12**

1 Introduction

1.1 The problem

Stablecoins are already the settlement layer of on-chain finance. USDC, USDT, and assets such as DAI and PYUSD quote prices, back loans, settle payments, and route RWA subscriptions. They can play that role because they are public assets on Ethereum: they form pairs with arbitrary tokens and are held directly by existing contracts.

A public ledger creates two problems:

1. **Ledger transparency.** Balances and transfers are visible to anyone. Treasury positions, market-maker flows, and fund counterparties can be traced; payroll and supplier settlements are equally unsuitable for a public book.

2. **Issuance equals claim.** Once minted, a stablecoin can be transferred and redeemed. Reserves sit with the issuer and its banks; whether circulating supply matches reserves depends on disclosure and audits. Users trust the issuer, not a checkable contract state.

A private stablecoin should move holders, amounts, and counterparties off the public ledger while still allowing selective disclosure to regulators or auditors. It should also be held the way today's stablecoins are held: the user pays fiat and receives this coin—not a public USDC balance that is later sent into a pool to be hidden. The coin must then trade against public assets and remain inside Ethereum DeFi. Issuance and redemption should no longer depend on trusting an issuer. Moving privacy to another chain means abandoning this settlement layer (Section 2).

1.2 Core ideas

1. **Payer ≠ recipient.** The party that locks USDC is not the party that receives the note. The user gives the ramp a payment address pa_U; the ramp produces the output note and commitment $\mathsf{cmx}_{\mathsf{out}}$, returns the note plaintext to the user, and signs `BuyAuth`. What appears on chain is the payer address and amount, not the user. Fiat delivery already requires trusting the ramp; who computes the commitment does not affect the on-chain binding: the $\mathsf{cmx}_{\mathsf{out}}$ signed into `BuyAuth` receives the entire payment.

2. **Minting ≠ becoming money.** Notes minted by the issuer must belong to a public protocol address pa_P—protocol notes (inventory). The circuit exposes value entering and leaving pa_P; the contract accordingly forbids inventory from participating in `transfer`. A protocol note becomes a user note (circulating) only through `buy` (equal USDC deposited in the same transaction), or returns to inventory through `redeem` (USDC withdrawn in the same transaction). Backing attaches at the sale, not at mint.

1.3 Contributions

1. **Permissionless issuance protocol.** Anyone can deploy a `ReserveVault` and `NotePool` backed 1:1 by a chosen ERC-20 reserve; pUSDC is the running instance with USDC reserves.

2. **Day-one privacy.** The payer locks the reserve token; the user's first on-chain credit is already a private note—never a public reserve balance that is shielded later. Later transfers are proved locally by the user and submitted by a relayer; the user's EOA need not appear.

3. **Trustless issuance, backing, and redemption.** Pre-minted notes are non-transferable. Only an equal reserve deposit increases circulating supply. Reserve balance ≥ `circulating` (in pUSDC, `ReserveVault.USDC` ≥ `circulating`) is maintained by the contracts. The holder self-proves to redeem; intermediaries cannot redirect funds, and the operator cannot block exits.

4. **Remain inside Ethereum DeFi.** Notes issued under the protocol are an Ethereum asset. Exchange against public protocols can be packed into one atomic transaction; the chain never sees the holder.

A comparison with public USDC is in Table 1 (pUSDC instance; abstract).

1.4 Organization

Section 2 covers related work. Section 3 gives the architecture and goals. Section 4 describes the protocol, main paths, and DeFi composition. Sections 5 and 6 treat security, privacy, propositions, and compliance freezes. Section 7 discusses limitations.

2 Related work

2.1 Shielded pools on Ethereum: public coin first, privacy later

Public stablecoins work as a settlement layer because they are first-class assets on Ethereum. For a private stablecoin to play the same role, both on-ramp and circulation must look like today's stablecoins—**privacy from day one: the user's first on-chain credit is already this private coin**—and it must then remain inside Ethereum DeFi, trading against public assets while holdings and payments stay hidden from the chain.

Tornado Cash, Railgun, Aztec, and Privacy Pools [7, 8, 9] cannot provide that on-ramp. They require the user to **already hold** a public stablecoin, then send the balance into a pool to obtain a private form. Today's on-ramp is different: fiat arrives and the user holds USDC directly. Shielded pools offer no such path. A private dollar can only be a post-processing of a public dollar.

Because the pool holds "already-shielded public coin," composition with DeFi must exit the pool, operate on the public surface, and optionally re-enter. The pool also does not issue dollars or enforce a 1:1 circulating–reserve match.

This protocol (illustrated by pUSDC) matches the fiat path: the payer spends the reserve token, the ramp produces a private note at the user's payment address, and the user holds it directly. Notes issued under the protocol stay on Ethereum and can later interact with public or in-privacy liquidity (Sections 4.7 and 4.8).

2.2 Private stablecoins on non-Ethereum chains

Private stablecoins already exist on Aleo [10, 11, 12, 13]. Mainnet USDCx is backed 1:1 by native USDC locked on Ethereum via Circle xReserve; USAD is issued by Paxos Labs against USDG. Minting is gated by roles such as `MINTER_ROLE`. Privacy holds only on Aleo; bridging back to Ethereum ends privacy. Trading against Uniswap or Aave becomes a cross-chain mint/burn. Aleo maps reserves through an issuer with minting privilege; this protocol lets anyone deploy a `ReserveVault`/`NotePool`, separates unpaid inventory from paid circulating notes, and constrains circulating supply in the contracts (pUSDC uses USDC as the reserve).

2.3 Comparison and Ethereum constraints

Table 2 summarizes the three approaches. A private stablecoin must live on Ethereum: pairing, lending, and custody already do.

Table 2: Related-work comparison (last column: deployable issuance protocol, illustrated by the USDC instance pUSDC)

	Shielded pools	Aleo	This protocol (pUSDC)
On-ramp	Hold public coin, then shield	Cross-chain deposit / role mint	**Fiat → private note**
Settlement layer	Ethereum	Aleo (public if bridged back)	**Ethereum**
Circ. ↔ reserves	No backing invariant	Issuer / bridge-side mapping	**Contract-enforced 1:1**
Mint gating	N/A	`MINTER_ROLE`	Inventory non-transferable; reserve required to circulate
Public DeFi	Exit pool, then interact	Cross-chain	**Same-chain atomic composition**

USDC's circulating–reserve match rests on issuer disclosure; the issuer can blacklist addresses [4]. The USDC-backed instance pUSDC inherits that credit and risk, but enforces circulating-side 1:1 in the contracts. A compliance freeze is this protocol's only on-chain privilege; it cannot move reserves away.

3 System overview

3.1 Architecture

Figure 1 lists components by layer; message flow is in Figure 2. Amber marks inventory, teal circulating value, and blue the Vault.

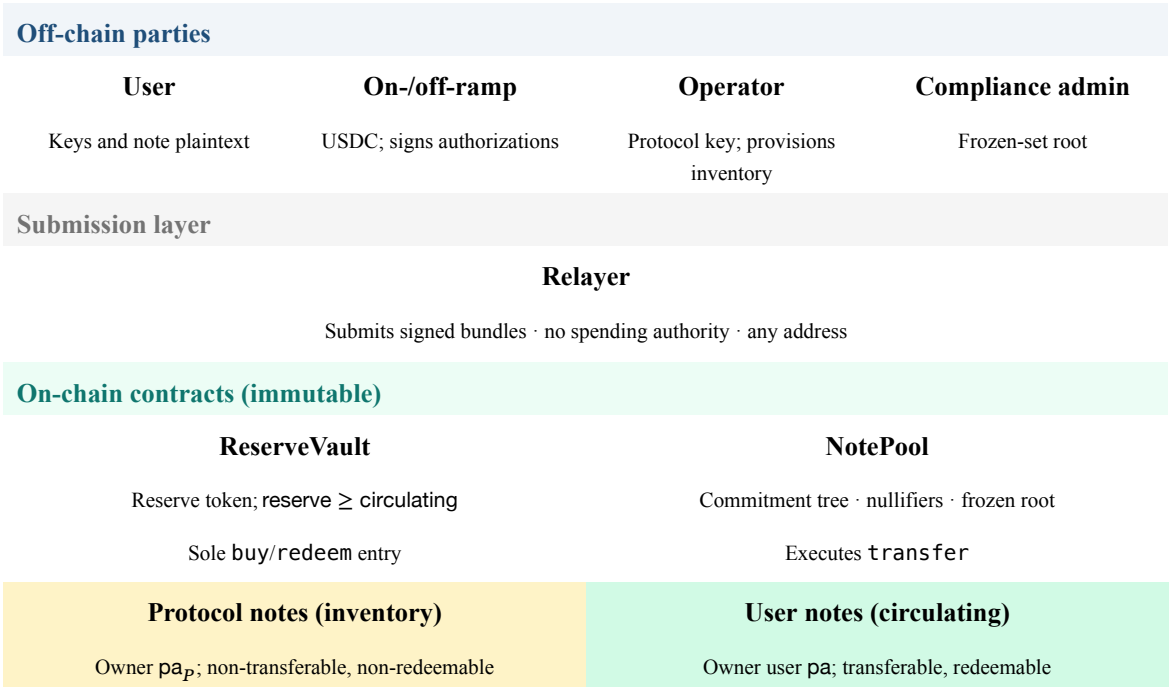


Figure 1: Issuance-protocol layered architecture (no arrows; pUSDC is the USDC-reserve instance). Components by layer; message flow in Figure 2.

Parties and contract roles are as follows (anyone may deploy a `ReserveVault`/`NotePool` pair).

- **User:** holds ask and note plaintext. At on-ramp, provides only pa_U; thereafter selects notes, proves, and signs locally. The long-term ask is never handed over.
- **On-/off-ramp:** holds the reserve token (USDC in the pUSDC instance). At on-ramp, produces $\mathsf{cmx}_{\mathsf{out}}$, signs `BuyAuth`, and returns plaintext; at off-ramp, receives the reserve and pays fiat.
- **Operator:** holds ask_P, `mint`s inventory, builds purchase bundles; holds viewing-key copies for read-only audit.
- **Relayer:** submits signed bundles; has no spending authority; any address may act as one.
- **Compliance admin:** updates the frozen-set root; cannot be rotated after deployment.
- **ReserveVault / NotePool:** the Vault holds the chosen reserve ERC-20; the pool maintains the commitment tree, nullifiers, and frozen root. The pool's `buy`/`redeem` accept calls only from the Vault contract (settlement and accounting are atomic in one transaction); the corresponding Vault entry points are public—anyone may call them without operator permission. Public `ReserveVault.redeem` is permissionless.

Contracts are non-upgradeable and cannot change verifier keys, fees, or submitter lists. Tree depth 32 is recommended; past half the leaves the pool enters exit-only mode, accepting only redemptions, so every unspent note can still be redeemed in full.

3.2 Two kinds of notes

Table 3: Two kinds of notes (circulating column highlighted)

	Protocol note (inventory)	User note (circulating)
Owner	pa_P (public)	User pa (in the commitment only)
Spend key	ask_P	User ask
`transfer`	**Forbidden**	**Allowed**
USDC backing	None	**Locked at purchase**

The pool tracks three quantities: `totalSupply` is the face value of all notes; `uncirculated` is the face value of protocol notes owned by pa_P; `circulating` is the face value of user notes, satisfying

$$\texttt{circulating} = \texttt{totalSupply} - \texttt{uncirculated}.$$

Inventory may exist ahead of reserves: `mint` increases only `totalSupply` and `uncirculated`, without moving USDC. Leaking ask_P cannot turn inventory into redeemable circulating value, but may leave inventory unsellable.

3.3 Trust and goals

We use three labels below: **A** = Assumptions, **G** = security Goals, **P** = Privacy goals. Later sections cite them by number.

Assumptions A (off-chain).

- **A6** (off-chain trust) The user trusts the chosen ramp and operator: the ramp issues a note at pa_U and signs `BuyAuth`; after off-ramp, it pays fiat. If the ramp builds $\mathsf{cmx}_{\mathsf{out}}$ as its own note, that is equivalent to never crediting the deposit. A6 does *not* enter the on-chain arguments.

Assumptions A (on-chain).

- **A1** The proof system is sound (correctness and knowledge soundness).
- **A2** Commitments are hiding and binding; signatures are unforgeable.
- **A3** Underlying USDC can freeze the Vault address (Circle blacklist inheritance).
- **A4** Ethereum consensus and single-transaction atomicity hold.
- **A5** Users keep their own ask secure.
- **A7** The compliance admin can update the frozen root (covering notes) but cannot transfer reserves.

Under A1–A5 and A7, G1–G6 hold against an arbitrarily behaving operator, ramp, and relayer.

Security goals G.

- **G1** $\texttt{USDC} \geq \texttt{circulating}$ (reserves cover circulating supply).
- **G2** Inventory cannot leave via `transfer`.
- **G3** Spending a user note requires the corresponding ask.
- **G4** USDC in a `buy` is drawn only from the `BuyAuth` signer, and $\mathsf{cmx}_{\mathsf{out}}$ receives the full a.
- **G5** A `redeem` pays only to the address or plan signed by the holder, for amount $a - \varphi$.
- **G6** Redemption needs no operator signature or allow-list; nobody can block or delay it except via a compliance freeze (submission goes through the public `ReserveVault`).

Privacy goals P.

- **P1** On-ramp need not involve a public address.
- **P2** `transfer` leaks neither party addresses nor amounts, and is unbound from the user's EOA.
- **P3** Privacy is against the public chain; the operator holding viewing keys is by design.

4 Protocol

4.1 Notes, actions, and signatures

pERC20 [3] follows the Zcash Orchard note model [1]. Note plaintext is $\mathsf{Note} = (g_d, pk_d, v, \rho, \psi, \mathsf{rcm})$. Commitments are hiding and binding; spends reveal a nullifier. A new note's ρ equals the nullifier of the note spent in the same action.

One action spends one old note and creates one new note; several actions form a bundle. Let V_{in} be the sum of spent face values and V_{out} the sum of newly created face values in the bundle. The public field

$$\texttt{valueBalance} = V_{\text{in}} - V_{\text{out}}$$

is the note-side net value of *this* transaction, constrained by the circuit and binding signature and written into SIGHASH. By convention: `valueBalance` < 0 means this transaction *creates* value on the note side (e.g., `mint`); > 0 means it *destroys* value (e.g., `burn`); $= 0$ means internal note conservation (`buy`/`redeem`/`transfer`). It differs from the pool's global counter `totalSupply`: the latter is contract state, updated by the entry point after success; the former describes only this bundle.

All signatures cover the same digest SIGHASH (nullifiers, commitments, ciphertexts, `valueBalance`, `recipientMeta`, `executor`). Without ask and the binding key, the digest cannot be altered. When `executor` $\neq 0$, only that address may submit.

4.2 Entry points and authorizations

Each entry point *hard-codes* the allowed `valueBalance` and `executor` into verification: callers cannot choose the net value, or inventory could become circulating without paying USDC. Table 4 lists the values; e.g., `mint`(a) creates face value a with no spends, so `valueBalance` $= -a$, after which the contract sets `totalSupply` $+= a$ and `uncirculated` $+= a$.

Table 4: Pool entry points (main-path `buy`/`redeem` highlighted; "Vault only" = pool-side msg.sender is `ReserveVault`)

Entry (pool)	Pool msg.sender	valueBalance	Notes
`mint`(a)	issuer	$-a$	Inventory only; no USDC
`burn`(a)	issuer	$+a$	Destroy inventory
`buy`$(a, \mathsf{cmx}_{\text{out}})$	Vault only	0	Inventory net $-a$; pull a USDC in same tx
`redeem`(a)	Vault only	0	Inventory net $+a$; pay $a - \varphi$
`transfer`	anyone	0	Zero inventory in/out

"Vault only" is not a privileged role: the external entry points are the public `ReserveVault.buy`/`redeem`; the bundle's executor is also fixed to the Vault, preventing a bypass that calls the pool without settling.

Who pays, signs the destination—paired with the `buy`/`redeem` entry points. The two algorithms differ.

`BuyAuth` (USDC→note). The payer signs an EIP-712 struct:

$$\mathsf{BuyAuth} = \mathsf{Sign}_{\mathsf{payer}}(\mathsf{EIP712}\{\mathsf{payer}, a, \mathsf{cmx}_{\text{out}}, \mathsf{nonce}, \mathsf{deadline}\}).$$

EOAs use **secp256k1 ECDSA** (on-chain `ecrecover`); contract wallets use **ERC-1271**. On the main path the ramp produces $\mathsf{cmx}_{\text{out}}$ and then signs. `ReserveVault.buy` verifies the signature and `deadline`,

marks (payer, nonce) used, then `transferFrom`(payer, `Vault`, a); circuit CT ensures $\mathsf{cmx}_{\mathsf{out}}$ receives the full a.

`RedeemAuth` (note→USDC). This is *not* a second EIP-712 object, nor a separate signature scheme. The holder writes a recipient digest into the bundle:

$$\mathsf{redeemDigest} = H(\mathsf{DOMAIN}_{\mathsf{REDEEM}}, \mathsf{recipient}, a),$$
$$\mathsf{recipientMeta} = H(\texttt{ReserveVault}, \mathsf{redeemDigest}).$$

then signs the same SIGHASH covering recipientMeta with the note's **SpendAuth** (Orchard spend authorization, randomized public key rk) and **binding signature**. When calling `redeem`, (recipient, a) are passed as *public parameters*—redemption exits the pool, so amount and recipient are public by nature. The `ReserveVault` contract recomputes recipientMeta from `address(this)` and the (recipient, a) it is about to transfer, and compares against the signed field in the bundle; a relayer that changes address or amount breaks the digest and verification fails. The Vault address is hashed in to prevent cross-contract replay.

4.3 Circuit extensions: CP and CT

C1–C6 do not distinguish inventory from user notes. `ActionPusdc` adds:

CP (protocol in/out).

$$\mathsf{isP}_{\mathsf{in}} = (\text{old note owner} = \mathsf{pa}_P), \qquad vP_{\mathsf{in}} = \mathsf{isP}_{\mathsf{in}} \cdot v_{\mathsf{old}},$$
$$\mathsf{isP}_{\mathsf{out}} = (\text{new note owner} = \mathsf{pa}_P), \quad vP_{\mathsf{out}} = \mathsf{isP}_{\mathsf{out}} \cdot v_{\mathsf{new}}.$$

The contract aggregates $\texttt{protocolIn} = \sum vP_{\mathsf{in}}$ and $\texttt{protocolOut} = \sum vP_{\mathsf{out}}$.

CT (target output).

$$\mathsf{isT} = (\mathsf{cmx}_{\mathsf{new}} = \mathsf{cmx}_T), \quad \mathsf{isT} \cdot (v_{\mathsf{new}} - v_T) = 0, \quad \mathsf{isT} \cdot \mathsf{isP}_{\mathsf{out}} = 0.$$

On buy, $(\mathsf{cmx}_T, v_T) = (\mathsf{cmx}_{\mathsf{out}}, a)$, so the payment enters the named note in full.

4.4 Π_{buy}: purchase (main on-ramp path)

Inventory `mint`: the issuer outputs a protocol note of amount a to pa_P; USDC and `circulating` are unchanged. Figure 2 shows the main on-ramp path; the blue region is the atomic steps inside one transaction.

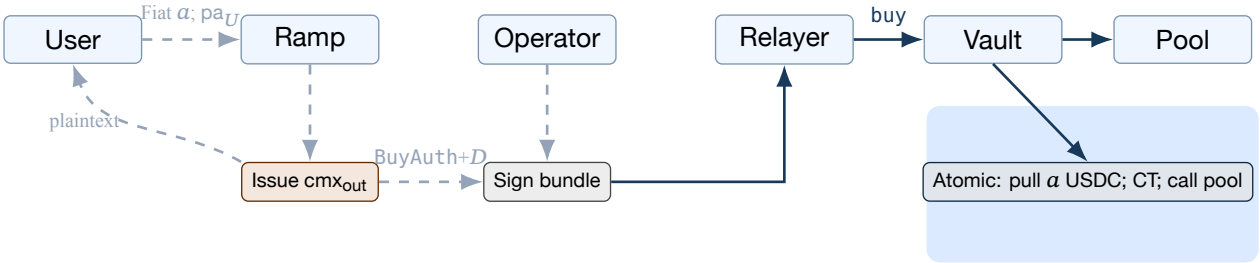


Figure 2: Π_{buy}: ramp issues the note and signs `BuyAuth`; operator signs inventory; Vault deposits atomically (blue).

1. Ramp issues: generates a one-time dummy and $\mathsf{Note}(\mathsf{pa}_U, a)$ (with $\rho = \mathsf{nf}_D$ fixing the uniqueness seed), obtains $\mathsf{cmx}_{\mathsf{out}}$, and returns plaintext to the user.
2. Ramp signs `BuyAuth`.
3. Operator spends inventory, pairs the dummy, proves and signs; spends further inventory for change.
4. Vault verifies, pulls USDC, and calls the pool; any failure reverts.

Visible on chain: payer, a, $\mathsf{cmx}_{\mathsf{out}}$, nullifiers. Invisible: user pa, user EOA.

4.5 Π_{transfer} and Π_{redeem}

Figure 3 shows transfer and redemption together. Teal marks the private-transfer atomic step; blue marks the redeem atomic step.

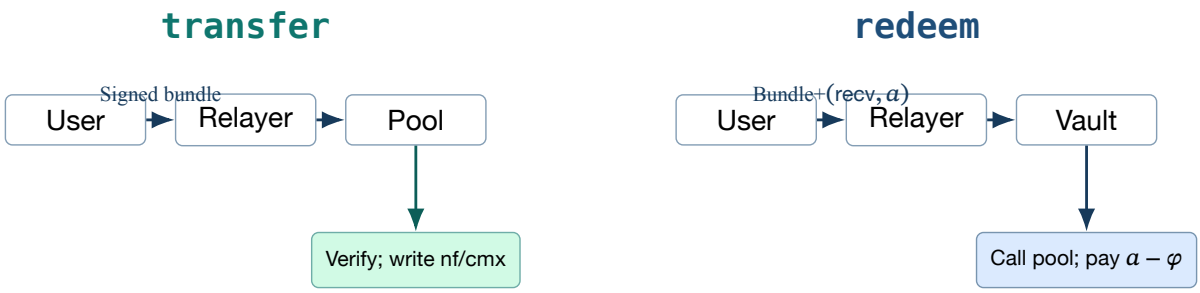


Figure 3: Π_{transfer} (left, teal) and Π_{redeem} (right, blue). Color blocks are atomic steps within one transaction.

`transfer`: when `executor` $= 0$, anyone may submit. Relayer compensation is a tip note in the same bundle. `redeem`: the output protocol note is the receipt; ask_P is not required. The holder or any relayer calls the public `ReserveVault.redeem`; the Vault then calls the pool. For fiat off-ramp, recipient is the off-ramp.

4.6 Responsibility matrix

Table 5: Responsibilities by flow (`buy` row highlighted)

Flow	Ramp	Operator	User	Relayer
`mint`	—	Build and submit	—	—
`buy`	Issue note; sign `BuyAuth`	Sign inventory	Provide pa_U	Submit
`transfer`	—	—	Select notes; sign	Submit
`redeem`	Receive USDC on off-ramp	—	Sign order	Submit
DeFi	—	Counter-sign `buy`	Sign plan	Submit

4.7 Building Private DeFi

A private stablecoin can either `unshield` into existing public liquidity (Section 4.8) or power applications that trade against existing liquidity without redeeming first. The latter splits by counterparty asset form:

1. **Private stablecoin ↔ public asset:** construct a one-step trade in which the private and public legs settle in the same transaction—not `redeem`/`unshield` then enter a public protocol.
2. **Private stablecoin ↔ already-shielded public asset:** both sides are private notes (e.g., a native private asset and a stablecoin that entered the pool via shield); matching and settlement stay entirely in the private domain.

PLabs ships a Mode 2 product prototype: **PEX** (on-chain private exchange). The current pair is **P20/sUSDC**: P20 is a native private asset issued under the pERC20 protocol; sUSDC is a note of USDC after shield into the privacy pool. The two trade non-custodially on a CLOB. Experience: `https://app.plabs.online/pex`.

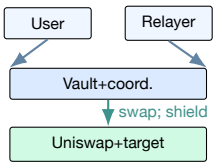
Mode 2's matching and settlement skeleton extends readily to Mode 1: replace one side with a public-asset leg that settles against a public balance or pool in the same transaction; the remaining non-custodial constraints are unchanged. This path is independent of the "redeem → public action → return to privacy" route in Section 4.8.

4.8 Composition with public DeFi

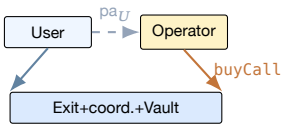
Unlike the one-step trades of Section 4.7, here the private stablecoin is first redeemed to a public ERC-20, then enters existing protocols such as Uniswap or Aave, and optionally `buy`s back into privacy. Public

protocols recognize only ERC-20. The holder packs "redeem → public action → return to privacy" into one transaction. Returning to pUSDC is a `buy`: the user provides pa_U; the operator issues the note and pre-signs `buyCall`.

Components include `redeemForDefi`, a non-reentrant coordinator, whitelisted Adapters, and a target privacy pool; planDigest and the redeem-order domain tags are disjoint. Properties: atomicity; plan binding; Adapters do not forward arbitrary calls; leftovers land only at a public address; sandwich loss is bounded by slippage limits.



Forward: pUSDC→pWETH



Reverse: pWETH→pUSDC

Figure 4: Composition with public DeFi, forward and reverse. Blue/teal: on-chain atomic execution; amber: inventory-related pre-signed `buy`.

5 Security and privacy

We discuss security goals under A1–A5, A7, and immutable deployment.

Every legitimate entry point leaves $\mathtt{slack} = \mathtt{USDC} - \mathtt{circulating} - \mathtt{feesAccrued}$ unchanged; each entry ends by asserting that slack has not regressed. Hence G1. Inventory cannot leave via `transfer` (G2). `BuyAuth`+CT pins $\mathsf{cmx}_{\mathsf{out}}$ (G4). Redemption is proved locally without the operator (G6).

Table 6: Public observables ("invisible" column highlights the privacy boundary)

Flow	Visible on chain	Invisible on chain
`buy`	Payer, a, $\mathsf{cmx}_{\mathsf{out}}$, nullifiers	User pa, EOA
`transfer`	Nullifiers, output commitments, relayer	Both pa, amounts, EOA
`redeem`	recipient, a, nullifiers	Spent note's pa and prior amount
Aggregates	`circulating`, inventory, Vault balance	Per-user balances

Three leakage channels. (1) Time correlation of rare-amount buy/redeem. (2) The payer address is public, but is not the user. (3) A colluding ramp and operator can tie KYC to pa—a deliberate concession to compliance.

5.1 Propositions and attack matrix

Under A1–A5, A7, and immutable deployment, the following propositions hold against an arbitrarily behaving operator, ramp, and relayer. Write $\mathtt{slack} = \mathtt{USDC} - \mathtt{circulating} - \mathtt{feesAccrued}$.

Proposition 1 (I1). At all times $\mathtt{circulating} = \mathtt{totalSupply} - \mathtt{uncirculated}$: both sides are maintained by the contract from value entering and leaving pa_P, so the equality holds by definition.

Proposition 2 (I2′). Every legitimate entry point leaves `slack` exactly unchanged, and asserts at the end that `slack` has not regressed. Hence while $\mathtt{slack} \geq 0$, no entry can break $\mathtt{USDC} \geq \mathtt{circulating} + \mathtt{feesAccrued}$; when a shortfall already exists (A3), entries must not deepen it, but redemptions remain allowed.

Proposition 3 (G1). The only entry that increases `circulating` is `buy`, which deposits equal USDC in the same transaction and uses CT to pin $\mathsf{cmx}_{\mathsf{out}}$ to the full a. Hence circulating redeemable claims cannot issue without payment.

Proposition 4 (G2). `transfer` requires `protocolIn` = `protocolOut` = 0. Inventory cannot leave via transfer as a user note.

Corollary 1. Over-minting inventory via `mint` only increases `uncirculated`, not redeemable claims; fund safety does not depend on a minting cap. Tree-full risk is resolved by exit-only mode (Section 3).

Proposition 5. Leaking ask_P cannot turn inventory into redeemable circulating value without paying USDC (Propositions 3 and 4). A holder can at most self-pay USDC to purchase; availability-wise they may rewrite inventory ciphertexts so the operator can no longer sell, so ask_P should be held under threshold signing.

Proposition 6 (G3). Spending a user note requires SpendAuth under the corresponding ask. Viewing keys (including the operator's FVK) do not contain ask and cannot spend or alter a signed SIGHASH.

Proposition 7 (G4). `buy` draws a only from the verified `BuyAuth.payer`; (payer, nonce) is single-use; CT forces an output of amount exactly a at $\mathsf{cmx}_{\mathsf{out}}$ that is not owned by pa_P. The payer cannot be swapped for another, and the receiving note cannot be intercepted or replaced.

Proposition 8 (G5). For `redeem`/`redeemForDefi`, the recipient address (or plan) is written by the holder into signed fields; the Vault (or coordinator) recomputes the digest from the public parameters it is about to execute and compares. Changing address, amount, or plan fails verification.

Proposition 9 (G6). Redemption proofs are generated locally; no operator signature is required. The external entry `ReserveVault.redeem` is public: any relayer (or the recipient itself) may submit; pool-side `onlyReserveVault` only requires "settlement must go through the Vault," not an allow-list. Nobody can block or delay except via a compliance freeze (A7); freezes cannot move reserves.

Table 7: Attack matrix

#	Attack	Mitigation	Basis
S-1	Move inventory as circulating	`transfer` forbids inventory I/O	Prop. 4
S-2	Issue circulating without payment	Fixed `valueBalance`; deposit in same tx	Prop. 2,3
S-3	Misreport protocol value I/O	CP computed in circuit	A1
S-4	Forge pa_P	On-chain check of pool constant	—
S-5	Redirect redeem destination	Vault recomputes digest	Prop. 8
S-6	Censor transfer/redeem	Permissionless entry points	Prop. 9
S-7	Fail to deliver fiat	Off-chain trust	A6
S-8	Infinite `mint` fills tree	Exit-only mode	Cor. 1
S-9	Buy into own note / skim	`BuyAuth`+CT	Prop. 7
S-10	Replay `BuyAuth`	Nonce + deadline	Prop. 7
S-11	Tamper signed bundle	Full SIGHASH coverage	C3,C5
S-12	Spend with FVK	ask not in FVK	Prop. 6
S-13	Double spend	Nullifier set	C2
S-14	Spend frozen note	C6	§6
S-15	Front-run dummy spend	Only voids that purchase	Π_{buy}
S-16	Sandwich DeFi swap	Slippage bounds	§4.8
S-17	Fee seizure blocks redeem	φ deployment constant	Table 4
S-18	Donate tokens to coordinator	End-of-tx balance sweep	§4.8
S-19	Nested ERC-1271 call	Non-reentrancy	§4.8
S-20	Swap verifier key / upgrade	No such privilege	§3
S-21	Treasury blacklist blocks redeem	Redeem never pays treasury	Table 4
S-22	Shortfall reverts all redeems	No-regression slack assert	§5
S-23	Tree full, cannot redeem	Exit-only reserves leaves	§3
S-24	ask_P leak rewrites inventory	No fund impact; threshold sig	Prop. 5

Collusion between operator and relayer still cannot issue circulating claims without payment or redirect funds. Operator and compliance admin can freeze notes but cannot move reserves. Collusion with the USDC issuer is outside the model (A3).

6 Compliance and auditability

The compliance admin maintains the frozen-set root. A spend must prove the commitment is not in the frozen set. Freezes are by `cmx`; value still counts toward `circulating`; reserves cannot be transferred. The contract does not verify root correctness; scope is decided by the admin. Viewing keys held by the operator are read-only. `uncirculated`, `circulating`, and Vault balance are publicly reconcilable on chain.

Table 8: Three note-related structures

Structure	Question	Proof	Valid root
Commitment tree	Exists?	Membership (C1)	Any historical root
Nullifier set	Already spent?	Contract lookup (C2)	—
Frozen set	Frozen?	Non-membership (C6)	Current root only

The frozen set is an indexed Merkle tree [6]. A malformed root can halt all spends; it cannot move reserves.

7 Discussion and limitations

Underlying USDC risk is inherited from the issuer. Fiat and inventory sales have no on-chain escrow. Immutability means defects require migrating the pool. Self-pay on-ramp degenerates the entry into an ordinary shield. Open questions: Adapter allow-lists, freeze governance, and exit-entry specs for other privacy pools.

8 Conclusion

We present an issuance protocol for private stablecoins on Ethereum: anyone can deploy a `ReserveVault` and `NotePool` backed 1:1 by a chosen reserve ERC-20. Instantiated with USDC reserves, that deployment is pUSDC; holdings are pERC20 notes. Inventory and circulating value are distinguished by owner; equal reserve payment is required to circulate; reserve balance $\geq$ `circulating` is checkable. After on-ramp, the user holds a private note directly. Who pays signs the destination. Shielded pools require a public coin first; notes issued under the protocol should be held like a stablecoin and remain inside Ethereum DeFi.

References

[1] Zcash Protocol Specification, Orchard.

[2] EIP-20; EIP-712; EIP-1271.

[3] Cyimon. Add ERC: Private Fungible Tokens (pERC20). ethereum/ERCs #1817, 2026.

[4] Circle, USDC on-chain supply and blacklist documentation.

[5] J. Groth. On the Size of Pairing-Based Non-interactive Arguments. EUROCRYPT 2016.

[6] Aztec Labs. Indexed Merkle Tree.

[7] A. Pertsev et al. Tornado Cash Privacy Solution. 2019.

[8] Railgun Project. Railgun Privacy System. https://docs.railgun.org.

[9] V. Buterin et al. Blockchain Privacy and Regulatory Compliance. 2023.

[10] Y. Fanusie, V.-L. Jaber, M. Green. Stablecoin Privacy. Aleo, 2026.

[11] Aleo. Private Stablecoins. https://docs.aleo.org/build/common-uses/private_stablecoins/.

[12] Circle. USDCx on Aleo Testnet via Circle xReserve. 2025.

[13] Aleo Network Foundation and Paxos Labs. USAD. 2025.